YOLO26n量化报告

YOLO26n UINT8 输入与 W8A8 混合量化报告

> **当前交付状态（2026-11-05 末）**
>
项	值
模型	YOLO26n，512×512 输入，one2one 双输出
输出 0	类别分数 `[1,5376,80]` INT8（NHWC）
输出 1	框 `[1,4,5376]` FP16（cx,cy,w,h，模型像素）
精度（64 图留出集）	mAP50 **0.5909**，mAP50–95 **0.4336**
板端	run 8 ms，端到端 **46 ms**，30 fps 无丢帧
运行时 / 工具链	librknnrt 2.5.0 / rknn-toolkit2 2.3.2
>
> 第 1–13 节是 11 月 2 日单输出方案的历史记录，第 14–17 节记录后续迁移（双输出、全 INT8 → FP16 框、512 输入）， 延迟剖析见 LATENCY_PROFILE.md。

报告日期：2026 年 11 月 2 日（Asia/Shanghai）。设备：Radxa ROCK 4D / RK3576。应用：autonomous-driving-car。

本文记录本次已完成的模型转换、校准、板端精度验证与问题定位，并说明去掉模型特征输出后的交付方案。数据来自本机固定验证集和保存的转换产物；不同阶段的模型和测试结果分别标明。

1. 交付结论

采用 UINT8 RGB 图像接口、INT8/W8A8 主干与 FP16 敏感检测头的混合量化方案。交付模型仅有一个检测输出 `output0`，形状 `[1,84,8400]`。按最新要求去掉 P4 特征输出、每目标 128 维向量、特征相似度关联及结果文件中的特征数据；目标跟踪保留检测框和运动学关联。

项目	原 FP16 模型	单输出混合量化模型
文件大小	10,528,118 字节	8,446,083 字节，减少约 19.78%
SDK 图像输入	UINT8、NHWC	UINT8、NHWC
查询到的模型输入	FP16 `[1,640,640,3]`	INT8 `[1,640,640,3]`
输入量化参数	不适用	scale=0.00392156886、zero-point=-128
检测输出	FP16 `[1,84,8400]`	FP16 `[1,84,8400]`
模型输出数量	1	1
模型特征输出	无	无，P4 实验输出已移除
固定验证集 mAP50	62.32%	61.20%
固定验证集 mAP50–95	47.18%	46.38%

这里的“u8 方案”指应用提交的 UINT8 图像缓冲。RK3576 内部使用 INT8/W8A8 路线，并保留部分 FP16 运算，不宜称为全模型纯 UINT8 量化。混合模型相对基线的 mAP50 与 mAP50–95 分别降低约 1.12 和 0.80 个百分点；数据范围和限制见下文。

2. 模型身份与工具环境

源权重来自 Ultralytics 官方 YOLO26n checkpoint，已核验 nano 架构以及原 FP16 模型的权重对应关系，详见同目录的 FRAME_PIPELINE.md。原 checkpoint 为 5,544,453 字节。

构件	SHA-256
官方 `yolo26n.pt`	`9b09cc8bf347f0fc8a5f7657480587f25db09b34bf33b0652110fb03a8ad4fef`
导出的 `yolo26n-pixel.onnx`	`6de676ce6144fbc6696f8d90e2f089d0718b54c5a09676bb7b3a01aaed6c93c2`
原 FP16 RKNN	`d4465c45a49e3ae995dd2dbb5e9a5984424aba3268ece02f112647db04f6963a`
交付的单输出混合 RKNN	`00c1c7c1a67a3a34dca32738267aa827eccd285a5023f174cdb07ac739491cfd`

环境项目	本次使用值
模型准备	Python 3.12、Ultralytics 8.4.187、Torch 2.2.0、ONNX、ONNX Runtime、onnxslim
模型编译	RKNN-Toolkit2 2.3.2，目标平台 `rk3576`
板端运行库	RKNN Runtime 2.3.0，构建标识 `c949ad889d@2024-11-07T11:35:33`
板端 NPU 驱动	0.9.8
相机配置	1280×720 MJPEG，配置目标 30 FPS
模型输入	640×640 RGB，静态 batch=1

模型准备使用隔离 Python 环境，正式 C/C++ 应用不依赖这些 Python 包。曾在独立目录对照官方 Runtime 2.3.2，用于排查量化输出问题；没有因此替换系统的 Runtime 2.3.0。模型能否运行以本次板端实测为准，不能据此推定任意 Toolkit、Runtime、驱动组合兼容。

3. 量化策略与实际数据类型

配置为 `w8a8`、按通道权重量化、`normal` 校准算法、优化级别 3。先保留自动混合精度建议，再把检测头的敏感浮点张量与完整输出拼接链设为 FP16。强制头部张量 61 项，叠加自动建议与拼接链后，`custom_quantize_layers` 共 67 项。

```
configuration = dict(
    target_platform="rk3576",
    mean_values=[[0, 0, 0]],
    std_values=[[255, 255, 255]],
    quantized_dtype="w8a8",
    quantized_algorithm="normal",
    quantized_method="channel",
    float_dtype="float16",
    optimization_level=3,
)
```

编译产物的实际参数统计如下。INT8 权重比例按权重元素数计算，不代表算子数、激活数或模型文件字节占比。

参数类别	张量数	元素数	数据类型
量化卷积权重	76	2,242,480	INT8
量化卷积偏置	76	7,192	INT32
保留浮点的权重	26	157,680	FP16
保留浮点的偏置	26	1,580	FP32

INT8 权重占比为 `2,242,480 / (2,242,480 + 157,680) ≈ 93.43%`。统计保存在 `build/actual-quantization.json`。

应用向 SDK 传入 UINT8、NHWC 图像，`pass_through=0`；让 SDK 依据模型输入属性完成转换。模型原生输入为 INT8、zero-point=-128、scale≈1/255。应用不能直接把无符号像素缓冲当作有符号 INT8，也不能在 CPU 端先归一化后再重复施加 `std=255`。检测输出设置 `want_float=1`，由 SDK 将 FP16 输出转换为工程后处理使用的 float。

4. 校准数据、独立验证与预处理

校准集共 96 张，包含 64 张 COCO128 图像和 32 张实际相机原始帧。相机帧来自未画框的 `/video/mjpeg`，接收样本间隔至少 0.5 秒，并通过 64×64 RGB 缩略图的最小平均绝对差筛选以减少重复。该接收间隔不等同于物理曝光时间间隔。

独立验证包含另外 64 张 COCO128 图像，共 506 个标注、63 个有标注类别；另留 8 张新相机帧，用于输出一致性检查，不参与 mAP。COCO 图像按数值 ID 排序后，零起始偶数位置用于校准，奇数位置用于验证，两集合不重叠。

数据构件	数量或校验信息
校准图像	96＝64 张 COCO＋32 张相机帧
独立精度验证	64 张 COCO，506 个标注，63 类
相机一致性验证	8 张，无人工标注
`calibration.txt` SHA-256	`e8be446e3388dcdae65d9b22537b970c6e08c86c19cbeac39ab6adca99466790`
官方 COCO128 ZIP SHA-256	`61e5e3028863d8ffc3b81d6a514603954889f0edd5e4b44c4ce60b2da99aeb8e`

原始数据来自 Ultralytics 官方 COCO128 压缩包。官方包缺少 ID `000000000250` 和 `000000000508` 的对应标签，另有两个没有对应图片的标签；本次把缺失标签图明确作为空标注背景图，单独保存合成的空标签和处理记录，不修改官方原始文件。

预处理逐步与工程保持一致：JPEG 解码为 RGB；计算 float32 比例 `min(640/w,640/h)`；缩放尺寸按 `int(dimension*scale+0.5f)` 得到；采用 OpenCV `INTER_LINEAR`；整数居中，空白填 RGB 114；保留 UINT8 0–255，模型配置只施加一次 mean=0/std=255。64 张验证 PNG 已与应用原生解码和 letterbox 结果逐像素核对，完全一致。

COCO128 属于 COCO train2017 的小子集。上述 mAP 用于固定样本上的量化回归比较，不代表完整 COCO val2017 成绩或实际驾驶场景的泛化能力。8 张相机帧缺少人工标注，不能用于声称真实场景检测精度。

5. 模型导出与输出约定

本次保留工程原有 one-to-many 检测接口。导出时检测头设为 `end2end=False`、`export=True`、`format='onnx'`、`dynamic=False`，融合卷积与 BN，导出 opset12、静态 batch=1，再简化并进行 ONNX 模型检查。黑图上 Torch 与 ONNX 的最大绝对差约为 0.0022583。

单输出 `[1,84,8400]` 包括像素单位的 `cx,cy,w,h` 和 80 类概率，不附加独立 objectness。生产后处理沿用阈值过滤、分类别 NMS、逆 letterbox 和最多 32 个检测。评估程序则读取完整原始输出，单独使用评估阈值，避免生产截断影响精度对比。

官方导出器也提供 INT8 和混合精度支持，但当前 Ultralytics RKNN 导出实现 可在检测头前截断 NPU 图并把浮点解码尾部放到主机。本次交付保留工程已经验证的单个、已解码检测输出，采用本次固定转换脚本和模型哈希复现，不能直接把任意新版默认导出物覆盖到线上。

6. 精度验证结果

评估统一采用置信度下限 0.001、分类别 NMS IoU=0.7、最多 100 个检测、IoU 0.50–0.95、101 点 AP。独立 NumPy 评估器读取原始浮点输出，并在模型坐标系先执行 NMS，再逆变换和裁剪。评估器以小型人工构造例验证过匹配和积分行为。

指标	FP16 基线	单输出混合量化	变化
mAP50	62.3239%	61.2043%	-1.1196 个百分点
mAP50–95	47.1768%	46.3817%	-0.7951 个百分点
高置信度检测数	188	182	同一对照阈值
同类、IoU≥0.5 匹配数	188 个作参照	179	保留约 95.21%
匹配框平均 IoU	—	0.98037	相对 FP16 输出
匹配置信度平均绝对差	—	0.01873	相对 FP16 输出
匹配置信度绝对差 P95	—	0.06855	相对 FP16 输出

本次重新评估的精度证据为 `car-feature-removal/evaluation.json`；与早期单输出候选的 `car-quant-u8/evaluation-fixed.json` 完全一致，均对应已经修复输出拼接链的单输出混合模型。上述 188/182 是独立评估器在置信度≥0.5 下的一致性统计；它们与生产后处理最多 32 个检测的计数口径不同。72 张生产后处理样本中，FP16 与量化模型分别得到 168 和 160 个检测。

7. 板端性能与测量口径

本次移除特征后，重新暂停在线推理，在同样的 72 张图像上串行测试 FP16 与单输出混合模型，固定 CPU4、NPU core0、一个上下文，首图预热两次，每图计时一次。两份逐图原始记录均保存在新的 `car-feature-removal/` 目录，未覆盖此前资料；查询确认两模型都只有一个 FP16 检测输出。

阶段	FP16 均值	单输出混合量化均值
`rknn_inputs_set`	16.044ms	0.381ms
`rknn_run`	50.417ms	46.554ms
`rknn_outputs_get`	2.700ms	2.703ms
检测后处理	0.696ms	0.720ms
上述四阶段合计	69.856ms	50.358ms
JPEG 解码，单独列出	2.766ms	2.752ms
letterbox，单独列出	1.055ms	1.063ms

上述四阶段合计均值缩短约 **27.91%**。合计时间按每图四阶段相加后统计：中位数为 68.910→47.127ms，P95 为 71.632→49.506ms。P95 不等于各阶段 P95 相加；JPEG、缩放、渲染、视频编码和浏览器播放均未计入上述四阶段合计。

全部 72 个样本均纳入统计，没有剔除尖峰。FP16 的 `rknn_run` 最大约 125.32ms；混合模型有约 209.97 和 102.82ms 的两个尖峰。因此混合模型的 `run` 均值高于其 P95，这并非计算错误，也说明均值提升不能解读成每帧都更快或不存在尾延迟。

`rknn_run` 是包含设备等待的 API 墙钟时间，不能当作纯 NPU 计算时间。板卡温度、频率和其他系统活动会影响耗时，本次没有把一次 72 图串行测试当作长期尾延迟保证。单上下文结果也不能直接推算双 NPU 自适应并发吞吐，或摄像头曝光到屏幕显示的端到端延迟。

本节只使用本次单输出模型数据。历史带 P4 的第二轮测试仍保留在 `car-quant-u8/`，其特征图解量化和 ROI 提取成本不混入本节主表。可复核的新记录为 `car-feature-removal/fp16-results.jsonl`、`quant-results.jsonl` 和 `report-timings.json`。

8. 特征输出实验与本次移除

此前曾给缓存模型增加 P4 张量 `/model.19/cv2/act/Mul_output_0`，形状 `[1,128,40,40]`、INT8。其 128 来自该网络层的通道数；后处理把每个检测框映射到 P4，在裁去边缘 5% 的 ROI 内进行 4×4 双线性采样，逐通道平均并做 L2 归一化，得到每目标 128 维向量。它不是新增训练得到的独立身份识别层。

历史带 P4 模型为 8,447,185 字节，SHA-256 为 `1525b0d27cb1e8eb07da8f506990a52ebf440d3cc2ff6a1bf11c42bda28ef7a9`。增加已有特征输出没有改变固定验证集的检测精度，但中间特征未经身份/ReID 监督，同类不同目标也可能产生很高余弦相似度，不能当作通用身份保证。

按本次要求，交付重新采用已经过板端验证的单输出混合模型，不需要重新校准主干。应用同步移除特征图读取、ROI 向量、相似度匹配、API 向量和新增结果中的特征字段。跟踪保留目标类别、框位置/尺寸、恒速预测、预测框 IoU、中心位移门控和目标 ID；长时遮挡后没有外观重识别能力。

历史实验的 `feature-similarity.json`、`evaluation-final.json`、`feature-tests/`、旧结果文件仍作为历史资料保存，不代表当前应用仍输出特征。旧日报中的已有特征内容不追溯删除。

9. 应用接入与结果文件

运行路径为 `config/yolo26.rknn`，由现有构建和部署脚本按哈希同步。模型、校准图像和相机样本不纳入 Git，代码、中文报告和 PDF 纳入版本管理。

去掉特征后，新结果仍追加到 `recordings/detections/YYYYMMDD/result_YYYYMMDD.txt`。保留时间、数量、图像尺寸、焦距原始值、帧号 `seq`、`capture_ms`、目标索引 `index`、类别编号 `class`、类别名、置信度及坐标；不再生成 `feature_method`、`feature_dim`、`feature_valid` 和 `feature[...]`。

推理结果按值进入有界队列，由已有报告线程批量落盘，相机和推理线程不直接写文件。跟踪使用原始检测框估计速度，显示用的平滑放在跟踪之后。模型重载清空跟踪历史，避免不同运行会话之间错误沿用目标状态。视觉目标选择不会开启车辆跟随或电机。

10. 可复现流程

1. 校验官方权重哈希，按第 5 节导出并核对 ONNX 输出形状、像素坐标约定及 Torch/ONNX 数值误差。
2. 按第 4 节固定校准和验证清单；保留来源、图像哈希、缩放填充参数和缺失标签说明。对照应用预处理，不对输入重复归一化。
3. 使用第 3 节参数调用 `config`、`load_onnx`，执行 `hybrid_quantization_step1(dataset=..., proposal=True, proposal_dataset_size=1)`。保留 `.model`、`.data` 和原始 `.quantization.cfg`。

4. 在自动建议基础上，把 `/model.23/` 检测头中可量化的浮点激活及 `output0`、`output0-rs`、`output0_int8` 设为 `float16`。整数形状张量和常量不得盲目列入自定义浮点层。本次共保留 67 项。
5. 用与第一步完全一致的配置执行 `hybrid_quantization_step2`，再 `export_rknn`。配置中的 `custom_string` 也保持一致；实际完整配置在 `build/quantization-config.json`，不可只复制部分参数后混用旧缓存。
6. 查询实际输入、输出数量、形状、类型、量化参数；确保只有一个 FP16 检测输出。检查原始输出全部有限、概率范围和像素坐标约定正确，再进行固定集精度和性能对照。
7. 验证通过后，把模型放到工程 `config/yolo26.rknn`，运行以下项目脚本；部署前保留旧模型、二进制、网页、配置和模型同步基线。

```
bash scripts/build.sh -R
sudo bash scripts/deploy.sh -i
```

1. 核对源码与部署模型的 SHA-256、服务运行版本、相机/显示/编码计数递增、网页视频、录像、目标选择/停止、模型重载与结果文件格式。没有重新验证的现场功能不据此声称通过。

完整校准及混合误差分析本次约 23 分钟；固定配置下第二步编译实测约 31.6 秒。时间取决于环境，某个校准子阶段的完成比例不能代表整个任务完成比例。

11. 关键故障与处理依据

现象或尝试	实测结果与处理
只把公开检测头和输出保留 FP16	概率固定约 0.000122，64 张验证 mAP 为零，候选拒绝部署
手工构造原生 INT8 输入	未修复异常，排除简单输入类型转换原因
绕过 SDK，读取原生 FP16 输出	异常仍在，排除单纯输出浮点转换原因
独立加载官方 Runtime 2.3.2	异常仍在，系统运行库保持 2.3.0
检查中间转换和最终拼接	`output0-rs`、`output0_int8` 仍走 INT8，大坐标与小概率共用刻度，概率分辨率丢失
将完整拼接链保留 FP16	概率恢复，重新通过全部固定集板端精度对照
盲目把形状/常量转浮点	`Constant_12_output_0` 等操作数不合法；仅筛选可量化浮点激活
第二步修改附加说明字符串	与缓存配置不一致导致构建中止；恢复第一步完整配置后通过

最终方案保留了解码和输出的数值范围，降低了主干及输入转换成本。未采用输出概率已经损坏的“更纯整数”候选。

12. 部署核验与回滚要求

本次移除特征后已通过项目脚本 Release 编译并部署。核验记录确认模型哈希为本报告的单输出混合模型；相机配置保持 1280×720，采样间隔内采集、显示与编码计数各增加 40，录像持续写入；跟踪 API 不再返回模型特征或外观对象，全部实际工作线程保持 FIFO，车辆跟随关闭。新一轮 72 图推理与 64 图精度评估也已完成，完整证据保存在 `car-feature-removal/`。该上线采样中没有检测目标，不把它作为真实目标关联或长时跟踪的精度证明。

回滚时一并恢复模型、对应应用二进制、网页、配置与 `config/.last-deployed.sha256` 同步基线；保持源码目录和部署目录的模型一致，再重启服务。原 FP16 模型保留在 `/home/qianruo/kbuild/car-quant-u8/original-fp16.rknn`，首次切换前的完整备份位于 `/home/qianruo/kbuild/car-quant-before-gUqtEH/`，本次移除特征前的完整备份位于 `/home/qianruo/kbuild/car-quant-before-sUFfJS/`。单输出混合模型的源产物位于 `car-quant-u8/build/yolo26n-w8a8-mixed.rknn`。

不启动车辆电机来验证视觉修改。现场 CAN 总线、电机闭环、完整 COCO 验证、长时真实道路与跨摄像头身份识别均不在本次量化验证范围内。

13. 证据与官方资料索引

下表本机路径统一以 `/home/qianruo/kbuild/` 为根。这些资料包含设备数据、模型转换依赖和临时工具，未整体提交 Git；报告保存可复核的关键参数、口径与哈希。

本机资料	内容
`car-yolo26n/provenance.md`、`comparison.json`	原权重身份、nano 架构和 FP16 权重对应核验
`car-quant-u8/data/README.md`、`manifest.json`、`checks.json`	来源、拆分、标签策略、预处理与校验
`car-quant-u8/data/calibration.json`、`validation.json`	图像清单、哈希、尺寸和几何变换
`car-quant-u8/build/export.py`、`export.json`	ONNX 导出及数值检查
`car-quant-u8/build/quantize-step1.py`、`quantize-step2.py`	校准、混合误差建议和第二步编译
`car-quant-u8/build/quantization-config.json`	两步共享的完整配置
`car-quant-u8/build/forced-fp16-head.json`、`actual-quantization.json`	浮点保留列表与实际类型统计
`car-quant-u8/build/candidate.json`	单输出候选哈希、大小和编译时间
`car-quant-u8/validation/infer.cpp`、`evaluate_raw.py`	原生板端验证与独立精度评估
`car-quant-u8/evaluation-fixed.json`、`hardware-validation.log`	单输出混合模型的精度及运行证据
`car-feature-removal/fp16-results.jsonl`、`quant-results.jsonl`	本次移除特征后，单输出模型逐图结果和分阶段耗时
`car-feature-removal/report-timings.json`	本次 72 图计时的均值与 P95 汇总
`car-feature-removal/evaluation.json`	本次重新完成的 64 图精度评估，与早期正确候选结果一致
`car-feature-removal/deploy.log`、`deploy-verification.json`	单输出部署、帧计数、录像、API 与实时线程核验
`car-quant-u8/evaluation-final.json`、`feature-similarity.json`	历史带特征输出实验，已从交付移除

官方参考：

- Rockchip RKNN-Toolkit2 v2.3.2：本次使用的工具版本与平台支持。
- Rockchip 模型转换示例：RK3576 使用 i8 路线及 mean/std 配置的示例。它是参考流程，不等同于本项目的完整混合方案。
- Ultralytics RKNN 导出源码：官方 INT8/浮点导出及检测解码尾部的组织方式。检索日期为 2026 年 11 月 2 日；当前主分支可能继续变化，复现本次结果应优先使用固定版本和哈希。

14. 2026-11-05：迁移到 one2one 双输出（全 INT8）

14.1 单输出方案的边界

`output0` 同时承载框坐标（校准集最大约 695）与类别概率（0–1）。INT8 仅 256 级，一组仿射参数覆盖 695 的跨度后每级约 2.72，0–1 的置信度会被压平，因此检测头必须保留 FP16。交付模型的 INT8 权重占比为 93.4%，检测头 61 层仍是 FP16，并非整网量化。

14.2 官方截断方案

Rockchip `rknn_model_zoo` v2.5.0 的 `examples/yolo26` 为 RK3576 给出另一条路径：按张量名截断 one2one 头，输出两支。

项目	取值
导出	`yolo export model=yolo26n.pt format=onnx`（默认 one2one 头）
截断张量	`/model.23/Sigmoid_output_0`、`/model.23/Mul_2_output_0`
转换	`rknn.load_onnx(model=..., outputs=[...])` + `rknn.build(do_quantization=True, dataset=...)`
输入	`(1,640,640,3)` INT8，scale=0.00392156886，zp=-128
输出 0	`(1,80,8400)` **INT8**，类别分数（已 sigmoid）
输出 1	`(1,4,8400)` **FP16**，框 cx,cy,w,h（模型输入像素，实测确认；FP16 的原因见 14.6）
交付哈希	`29cab9860993948e8636494f974ef0c4338cc4cd8030f49e2b3ada3d5875619b`

两支各自独立，类别分支用 INT8、框分支保 FP16：类别分数范围 0–1，8 位足够；框输出是像素单位，全 INT8 时每级约 2.73 px，是定位精度的主要损失来源（见 14.6）。

关于 NMS：NPU 内没有 NMS 算子，模型图在检测头结束。Rockchip 文档所说的"无需 NMS"指 one2one 头的设计意图；本项目实测该头在 conf≥0.60 时仍有 **85.7%** 的框在同类 IoU>0.45 下互为重复（64 图、956 框、单图最多 105 框），因此板端后处理**保留类内贪心 NMS**。去掉 NMS 会使 mAP50 从 0.5238 降到 0.3491。

框格式注意：`Mul_2` 输出是像素单位 `cx,cy,w,h`，不是 `x1,y1,x2,y2`。官方 C 示例按 xyxy 解释，若直接照搬会得到负宽高——本项目按 cxcywh 解码，偏差实测 0.5 个百分点内。

14.3 工具版本约束

本次仍用 RKNN-Toolkit2 **2.3.2**（aarch64 wheel + Python 3.12）。2.5.0 在 `hybrid_quantization_step1` 的图融合阶段崩溃：

```
File "rknn/api/rules/matmul.py", line 1539, in
_p_fuse_exmatmul_add_mul_exsoftmax13_exmatmul_to_sdpa
IndexError: list index out of range
```

640 与 512 输入、`optimization_level` 0/1/3 均可复现，是该模型的注意力结构与 2.5.0 融合规则的冲突，与输入尺寸无关。升级到 2.5.0 前需确认该规则已修复。

14.4 板端后端（纯 C）

- `yolo_postprocess_two()`：按两路各自的 zp/scale 反量化，逐锚点取最大类，阈值过滤后维护 128 个候选池，再做类内贪心 NMS（复用单输出路径的 `yolo_nms_finish`），最后取前 32。
- 推理请求 `want_float=0`，直接消费原生 INT8 输出，省去每帧的 FP16→float 转换。
- 单输出路径完整保留：混血 FP16 模型仍可加载运行，作为回滚路径。

14.5 板端实测（640 输入，NPU 1188 MHz）

指标	单输出混合	双输出全 INT8
`run`	22.2 ms（Runtime 2.3.0）	19–21 ms（2.3.0）→ **12–13 ms（2.5.0）**
`outputs`	2 ms	0 ms（原生 INT8）
`post`	含 NMS	2 ms，含 NMS（候选池 128）
采集帧率	30 fps	30 fps
运行错误	—	0 条

14.6 精度验证（2026-11-05 完成）

同一批 64 图留出集、同一套解码口径（置信度 ⩾0.001、类别内 NMS IoU 0.7、101 点 AP）：

模型	mAP50	mAP50–95
单输出混合（旧交付）	0.5005	0.3705
双输出全 INT8	0.5238	0.3390
双输出 INT8 + 框输出 FP16（现交付）	**0.5471**	**0.3635**
相对全 INT8	**+0.0233**	**+0.0245**

框输出改 FP16 的收益来自量化步长：全 INT8 时框的 scale 为 2.7269，即每级 2.73 像素、±1.4 px 误差；FP16 后该误差消失，而代价仅是每帧多传 33 KB（4×8400 个数由 1 字节变 2 字节），板端 `outputs` 耗时仍在 0–1 ms。

mAP50 提升 2.3 个点，mAP50–95 下降 3.15 个点：one2one 头在高 IoU 下的定位精度略弱，召回更好。按既定门限（mAP50 降幅超过 2 点回滚）不触发回滚。

同一批数据在**关闭 NMS** 的口径下复测：旧模型 mAP50 降至 0.2973、新模型降至 0.3491，即 NMS 对本次交付模型价值约 **17.5 个点**。这也否定了"one2one 头可免 NMS"的说法。

评测方法：C 侧把两路 INT8 输出反量化后按 `[84,A]` 落盘（0–3 行为 cx,cy,w,h），复用既有评分脚本，与历史结果同口径；不含 Python 推理环节。

14.7 512 输入实验（未部署）

同流程在 512×512 下实测：`run` 17.42 ms（640 为 24.04 ms）、总耗时 19.90 ms（640 为 28.74 ms），输出 `[1,84,5376]`。该模型是一次性实验产物，未部署（精度未验证，且当时单输出方案仍在服役），已于 2026-11-05 清理删除；如需复现，按 15.1 的流程把导出与校准集尺寸换成 512 重跑即可。

14.8 回滚

迁移时的单输出模型备份已于 2026-11-05 随旧文件清理一并删除。当前二进制同时支持单输出与双输出两种模型， 回滚只需重新产出一份单输出模型（流程见第 11 节）并替换 `config/yolo26.rknn`：

```
sudo cp <single-output>.rknn /opt/autonomous-driving-car/config/yolo26.rknn
sudo systemctl restart autonomous-driving-car
```

运行时无需降级：`librknnrt` 2.5.0 向前兼容 2.3.2 编译的模型。

15. 按官方 2.5 流程重做与实测偏差（2026-11-05）

15.1 官方流程原文

`rknn_model_zoo/examples/yolo26` 给出的步骤只有两步：

```
# 1. YOLO 导出 ONNX
pip install ultralytics
yolo export model=yolo26n.pt format=onnx

# 2. RKNN 官方工具转换
cd python
python convert.py <onnx_model> rk3576 i8 <output_rknn_path>
```

`convert.py` 内部只做三件事：`load_onnx(outputs=['/model.23/Sigmoid_output_0', '/model.23/Mul_2_output_0'])` 截断成两路，`build(do_quantization=True, dataset=...)` 量化，`export_rknn()` 导出。本次按原文执行（仅把 `DATASET_PATH` 指向本项目校准集），转换成功。

15.2 产物与线上模型等价性

项	官方 convert.py 产物	线上部署模型
输入	INT8, zp=-128, scale=0.003922	相同
输出 0（类别）	INT8, zp=-128, scale=0.003711	相同
输出 1（框）	INT8, zp=-128, scale=2.726881	相同
字节数	8,238,952	8,238,952

两者哈希不同仅因元数据，网络与量化参数逐项一致，线上模型即官方流程产物。

15.3 偏差一：框格式是 cx,cy,w,h，不是 x1,y1,x2,y2

官方 README 与 C 示例均按 `x1,y1,x2,y2` 解释 `Mul_2`，照此实现会得到负宽高、mAP 归零。实测对齐同一目标的数值可确认：

```
Mul_2 原始值 : (490.85, 321.8, 239.95, 270)
旧模型同目标 :  cx=482.8  cy=322.2  w=239.5  h=282.5
                ↑ 第 0 列≈cx        ↑ 第 2 列≈w
```

本项目按 `cx,cy,w,h` 解码，再 `x1=cx-w/2` 还原。

15.4 偏差二：文档说免 NMS，实测仍必需

官方称 one2one 头无需 NMS。本项目在 64 图留出集、应用阈值 conf≥0.60 下统计：**956 个框，85.7% 在同类 IoU>0.45 下互为重复，单图最多 105 个**。关闭 NMS 后 mAP50 由 0.5238 降至 0.3491（旧模型由 0.5005 降至 0.2973）。

NPU 内也没有 NMS 算子，模型图在检测头结束；`TopK`/排序抑制属主机侧逻辑。因此 C 后处理保留候选池 128 + 类内贪心 NMS + top32，耗时 2 ms。

15.5 偏差三：量化精度边界

张量	scale	含义
输入	0.003922	1/255，UINT8 像素
类别输出	0.003711	约 1/270 的分数分辨率
框输出	2.726881	每级约 2.73 像素，即 ±1.4 px 量化误差

框的量化步长是 mAP50–95 相对旧模型下降约 3 个点的主要原因；对 640 输入的定位精度影响可见但可接受。

15.6 运行时升级到 2.5.0

`librknnrt` 2.3.0 → 2.5.0 后，**同一模型**的 `run` 由 19–21 ms 降至 12–13 ms（约 25%，官方 release notes 的"优化多输入输出模型的 CPU 负载、修复内部内存复用"应验）。应用与现有模型无需改动，向后兼容。

15.7 工具链约束：2.5.0 无法转换本模型

升级到 toolkit 2.5.0 后，无论 `hybrid_quantization_step1` 还是官方 `build()` 路径、无论是否截断，均在同一处失败：

```
File "rknn/api/rules/matmul.py", line 1539,
  in _p_fuse_exmatmul_add_mul_exsoftmax13_exmatmul_to_sdpa
IndexError: list index out of range
```

640/512 输入、`optimization_level` 0/1/3 均可复现，属该模型注意力结构与 2.5.0 融合规则的冲突。因此**模型仍用 2.3.2 编译，运行时用 2.5.0**；升级工具链前需确认该规则已修复。

15.8 e2e（end2end）尝试与结论

YOLO26 的 `end2end=True` 分支在 checkpoint 中存在（`one2one_cv2/cv3`），导出 ONNX 可得 `(1,300,6)` 的免 NMS 输出。实测：

环节	结果
ONNX 推理	正确，框 `[369.4, 182.0, 603.3, 464.3]`，conf 0.908，cls 23
toolkit 2.5.0 转换	失败（同 15.7 的 IndexError）
toolkit 2.3.2 转换	成功，但真机推理框错误（`[167.8, -0.1, 207.0, 2.8]`），头部保 FP16 后仍错
2.3.2 + 不量化	可编译，未改变上述结论

说明 2.3.2 无法正确 lower `TopK`/`GatherElements` 这套 e2e 图，而支持这些算子的 2.5.0 又无法转换本模型。**结论：当前工具链组合拿不到可用的 e2e 模型**，需等待 2.5.x 修复融合规则，或改用官方预置 ONNX（仅提供 s 模型）。

15.9 结论

线上运行的就是官方流程产物：YOLO 导出 ONNX → RKNN 官方工具转成双输出全 INT8 模型，主机侧按 `cx,cy,w,h` 解码并做 NMS。三处与官方文档不符的地方均有实测依据，重跑时应以此为准。

16. 框输出 FP16 混合精度方案（2026-11-05 交付）

16.1 问题：框输出的量化步长

`Mul_2` 输出的是**像素单位**的 `cx,cy,w,h`，跨度约 0–640。整网 INT8 时该张量只有一组仿射参数：

```
scale = 2.726881  →  每级 ≈ 2.73 像素，量化误差 ≈ ±1.4 px
```

对 640 输入的检测器来说，±1.4 px 的定位误差会直接吃掉高 IoU 区间（mAP50–95 比头部 FP16 的旧模型低 3.15 个点）。

16.2 方案：只把框输出保 FP16

保留主干与类别输出的 INT8，仅把框输出张量标记为 FP16：

张量	类型	zero-point	scale
`images`	INT8	−128	0.00392156886
`/model.23/Sigmoid_output_0`	INT8	−128	0.00371079287
`/model.23/Mul_2_output_0`	**FP16**	—	—

代价只有传输量：该张量 4×8400 个数，由 1 字节变 2 字节，每帧多 33 KB。模型文件 8,238,952 → 8,242,011 字节（+3 KB）。

16.3 构建步骤

官方 `convert.py` 走的是 `rknn.build()`，没有按层指定精度的入口，因此该版本改用 hybrid 两步流程（工具链仍为 **2.3.2**，2.5.0 在本模型上会崩，见 15.7）：

```
OUT = ['/model.23/Sigmoid_output_0', '/model.23/Mul_2_output_0']

# step1：截断两支 + 生成量化配置
rknn.config(mean_values=[[0,0,0]], std_values=[[255,255,255]], target_platform='rk3576')
rknn.load_onnx(model='yolo26n.onnx', outputs=OUT)
rknn.hybrid_quantization_step1(dataset=calibration_txt, proposal=False)

# 修改生成的 quantization.cfg
custom['/model.23/Mul_2_output_0']    = 'float16'
custom['/model.23/Mul_2_output_0-rs'] = 'float16'
cfg['custom_quantize_layers'] = custom

# step2：按修改后的配置量化并导出
rknn.hybrid_quantization_step2(model_input=..., data_input=..., model_quantization_cfg=cfg)
rknn.export_rknn('yolo26n-box16.rknn')
```

`proposal=False` 是必要的：本模型的注意力结构在 `proposal=True` 时会触发 2.5.0 的融合 bug，2.3.2 下也可安全跳过。

构建日志会确认精度：

```
W hybrid_quantization_step2: The default output dtype of '/model.23/Sigmoid_output_0'
  is changed from 'float32' to 'int8' in rknn model for performance!
```

```
W hybrid_quantization_step2: The default output dtype of '/model.23/Mul_2_output_0'
  is changed from 'float32' to 'float16' in rknn model for performance!
```

16.4 板端 C 实现

推理侧按每路输出的实际 dtype 决定取值方式，不再假设整网同精度：

```
outputs[i].want_float = (nout == 1) ? 1
                      : (wk->out_attr[i].type == RKNN_TENSOR_FLOAT16 ? 1 : 0);
```

- INT8（类别分）→ `want_float = 0`，直接拿原生缓冲，由后处理按 zp/scale 反量化；
- FP16（框）→ `want_float = 1`，由运行时转成 float 缓冲。

后处理 `yolo_postprocess_two()` 按 dtype 选择读取路径，两种模型都能加载：

```
const int8_t *bp  = (ba->type == RKNN_TENSOR_INT8) ? (const int8_t *)boxes_buf : NULL;
const float  *bpf = (ba->type == RKNN_TENSOR_INT8) ? NULL : (const float *)boxes_buf;
float bcx = bpf ? bpf[i0] : yolo_deq_i8(bp[i0], b_zp, b_sc);
```

16.5 实测结果

板端（RK3576，NPU 1188 MHz，Runtime 2.5.0）：

项	全 INT8	框输出 FP16
`run`	12.2 ms	12.2 ms
`outputs`	0 ms	0–1 ms
`post`	2 ms	2 ms
模型体积	8,238,952 B	8,242,011 B
mAP50	0.5238	**0.5471**
mAP50–95	0.3390	**0.3635**

两项精度同时提升（+2.33 / +2.45 点），速度无可见变化，交付哈希 `ba7fc2c4…`。

16.6 备选方案对比

方案	框量化步长	mAP50	mAP50–95	结论
全 INT8（四通道共用 scale）	2.73 px	0.5238	0.3390	已弃用
框通道拆 4 个独立输出	2.51 / 2.73 / 2.62 / 2.63 px	未测	未测	**无收益，未采用**
框输出 FP16（现交付）	≈ 0	**0.5471**	**0.3635**	采用

拆 4 路之所以无效：量化范围由**校准数据的实际分布**决定，校准集中存在接近整幅的大框，w/h 的范围同样被顶到 640 附近，拆分后步长只从 2.73 降到 2.62，且输出数由 2 增到 5、取输出开销上升。该实验模型保留在 `build/yolo26n-5out.rknn`，未部署。

16.7 复现与回滚

复现：按 16.3 的脚本，校准集使用 `data/calibration.txt`（96 张，64 COCO + 32 实拍，640×640 letterbox），工具链 2.3.2。

回滚到全 INT8：把 `yolo26n-official.rknn` 一类全 INT8 模型放回 `config/yolo26.rknn` 即可，二进制同时支持两种输出精度（16.4 的双路径），无需改代码或降级运行时。

17. 512 输入迁移（2026-11-05 交付）

17.1 动机

640 方案里 `run` 占每帧 worker 时间（18 ms）的 67%，是唯一还有量级空间的一项；decode 受硬件限制、resize 需移植 RGA、post 至多省 1 ms。

17.2 构建

沿用第 16 章的流程（截断双输出 + `Mul_2` 保 FP16），只把导出与校准集尺寸换成 512：

项	640	512
锚点数	8,400	**5,376**
校准集	96 张 640 letterbox	96 张 512 letterbox
模型体积	8,242,011 B	7,888,539 B

17.3 精度（64 图留出集，同一口径）

模型	mAP50	mAP50–95
640 + INT8/FP16	0.5471	0.3635
512 + INT8/FP16（现交付）	**0.5909**	**0.4336**
差值	**+0.0438**	**+0.0701**

缩到 512 反而两项都涨，与"小输入损小目标"的常规预期相反。留出集来自 COCO128（图像多为 640×480），512 letterbox 后的有效目标尺度可能更接近训练分布；真实路况下的表现仍需观察，不能仅凭该数据集断言 512 普遍更优。

17.4 板端性能

指标	640	512
`run`	12 ms	**8 ms**
`resize`	4 ms	2–7 ms
decode→display	23–27 ms	**15–16 ms**
端到端 `processing_age`	61 ms	**54 ms**
采集 / 丢帧	30 fps / 0	30 fps / 0

`run` 的降幅（12 → 8 ms）与离线基准的比例（24 → 17.4 ms）一致；端到端省 7 ms。

17.5 运行参数来源

letterbox 尺寸取自模型输入张量的维度，`yolo_input_size` 配置项仅在模型未提供四维输入时作为回退。因此替换模型即切换输入尺寸，无需同步改配置；配置文件由应用启动时重写，不纳入版本管理。

17.6 回滚

把 640 版本模型放回 `config/yolo26.rknn` 并重启即可，二进制同时支持两种输入尺寸与两种输出精度（见 16.4 的双路径）。